

Penetration Testing Report

HTB – Oopsie

Executive Report

Target

Linux – Web Application

Assesment Type

Penetration Testing Laboratory

Prepared by

José Martín

Assessment Period

July – August 2026

CONFIDENTIALITY

Educational & Professional Portfolio

Index

1. Statement of Confidentiality.....	2
2. Engagement contacts.....	2
3. Executive summary.....	2
4. Approach.....	3
5. Assignment.....	3
6. Scope.....	3
7. Assessment summary.....	4
8. Detailed Walkthrough.....	4
8.1 Reconnaissance And Enumeration.....	4
8.2 Broken Access Control via Client-Side Cookie Manipulation.....	5
8.3 Insecure Direct Object Reference (IDOR).....	5
8.4 Unrestricted File Upload.....	5
8.5 Remote Code Execution.....	6
8.6 Credential Discovery And User Access.....	6
8.7 Privilege Escalation.....	6
8.8 Path Hijacking.....	7
8.9 The Flags.....	7
9. Remediation Summary.....	7
10. Appendix A : Finding severities.....	9
11. Appendix B: Tools.....	9
12. Appendix C : Technical report.....	10

1. Statement of Confidentiality

This report was produced as a part of a part of a personal penetration testing laboratory exercise.

The information contained in this document is intended solely for a educational and professional portfolio purpose. Any Ip Addresses, credential , flag and system information referenced herein belong to the target laboratory enviroment a must not used against unauthorized systems.

Distribution of this report should be limited for an educational, recruitment and professional development purposes.

2. Engagement contacts

Assessor Contact			
Assessor name	Title	Project	Assessor email
X	Project Director	Oopsie	X@htbcompany.htb
José Martín	Penetration Tester	Oopsie	Jose@htbcompany.htb

3. Executive summary

A penetration test was conducted against the Oopsie virtual machine from Hack The Box to identify vulnerabilities that could allow unauthorized access and privilege escalation.

The assessment identified multiple vulnerabilities affecting the web application and operating system configuration, including broken access control, insecure direct object references (IDOR), unrestricted file upload, remote code execution, exposed credentials, SUID misconfiguration, and PATH hijacking.

By chaining these vulnerabilities, full compromise of the target was achieved. Initial access was obtained through remote code execution as www-data. Application credentials were subsequently recovered and reused to access the robert account. Finally, a root-owned SUID binary was exploited through PATH hijacking, resulting in root-level command execution.

The assessment demonstrated that weaknesses in application authorization controls and local privilege management could be chained together to achieve complete system compromise.

4. Approach

The assessment followed the standard penetration testing methodology

1. Reconnaissance
2. Service enumeration
3. Web application enumeration
4. Vulnerability research
5. Initial exploitation
6. Post exploitation
7. Privilege escalation
8. Evidence collection

5. Assignment

Assignment			
From	To	Start	End
Project Director	José Martín	28/07/2026	12/08/2026

Target		
Name	Platform	Operating System
Oopsie	Hack the Box	Linux

The objectives are:

- Obtain initial access to the target system
- Escalate privileges to root
- Retrieve the available flags
- Document the attack path and identified vulnerabilities
- Provide remediation recommendations

6. Scope

In scope

- Linux operating system
- Web Application
- Local privilege escalation vectors

7. Assessment summary

Assesment summary	
Findings	Severity
Broken Access Control	High
Insecure Direct Object (IDOR)	High
File Upload	High
Remote Code Execution (RCE)	Critical
Exposed Credentials	Medium
SUID Misconfiguration	Critical
Path Hijacking	Critical

8. Detailed Walkthrough

The process had the following phases

8.1 Reconnaissance And Enumeration

Network reconnaissance identified two primary TCP services:

- **SSH**
- **HTTP**

Additional UDP enumeration identified a DHCP service.

The HTTP service was identified as an Apache web server running on Ubuntu and became the primary focus of the assessment.

Web enumeration using WhatWeb, Nmap scripts, Gobuster, Nikto, and manual inspection identified:

- PHP-based web application components
- /themes
- /uploads
- /cdn-cgi/login/
- An administrative authentication interface
- An administrator email address
- An exposed internal IP address
- Several low-severity web server configuration issues

A targeted review of the application's JavaScript references led to the discovery of the authentication interface under /cdn-cgi/login/.

No directly exploitable vulnerability was confirmed during the initial web server vulnerability scanning.

8.2 Broken Access Control via Client-Side Cookie Manipulation.

The authentication mechanism exposed the user's authorization level through the client-controlled role cookie.

The original session contained:

- user=2233; role=guest

Changing the role value to:

- user=2233; role=admin

resulted in access to the administrative interface.

This demonstrated that authorization decisions relied on client-controlled session data.

Impact: A low-privileged user could gain unauthorized access to administrative functionality.

8.3 Insecure Direct Object Reference (IDOR)

The administrative interface used user-controlled identifiers to retrieve client and account information.

By modifying the orgId and id parameters, resources belonging to other users could be accessed without appropriate authorization checks.

This allowed the administrator's Access ID to be identified and subsequently used during the file-upload attack chain.

Impact: Unauthorized access to sensitive client and account information.

8.4 Unrestricted File Upload

The administrative interface provided a file-upload functionality.

After obtaining administrative access, a legitimate PNG file was uploaded successfully and was subsequently located under the publicly accessible /uploads directory.

Further testing confirmed that PHP files uploaded to this directory were interpreted and executed by the web server.

Impact: The file-upload vulnerability could be escalated to server-side code execution.

8.5 Remote Code Execution

A PHP webshell was uploaded to the /uploads directory.

The webshell accepted commands through a GET parameter and successfully executed operating-system commands on the target.

The execution context was:

- uid=33(www-data)
- gid=33(www-data)
- groups=33(www-data)

This confirmed Remote Code Execution under the privileges of the web server process.

Impact: An attacker could execute arbitrary operating-system commands on the target system.

8.6 Credential Discovery And User Access

Following initial access, the web application's files were enumerated.

The file:

- /var/www/html/cdn-cgi/login/db.php

contained credentials that were reused by the robert user.

These credentials allowed the assessment to transition from:

- www-data → robert

The robert account was subsequently investigated for local privilege escalation opportunities.

8.7 Privilege Escalation

Enumeration identified:

- /usr/bin/bugtracker

with the following permissions:

- -rwsr-xr-- 1 root bugtracker ...

The SUID bit indicated that the binary executed with the privileges of its owner, root.

Further analysis of the executable revealed that it invoked:

- cat /root/reports/

without specifying an absolute path.

This created a potential PATH hijacking condition.

8.8 Path Hijacking

The robert user's PATH was modified to prioritize /tmp:

- `export PATH=/tmp:$PATH`

Command resolution confirmed:

- `which cat`
- `/tmp/cat`

The malicious /tmp/cat script executed:

- `#!/bin/bash`
- `/bin/bash -p`

When the SUID bugtracker binary subsequently invoked cat, the malicious executable was resolved through the manipulated PATH.

The resulting privileges were verified as:

- `uid=0(root)`

This confirmed successful privilege escalation to root.

8.9 The Flags

The user flag was accessible from

- `/home/robert/user.txt`

without requiring privilege escalation because the file was readable by other users.

After successful escalation to root, the root flag was retrieved from:

- `/root/root.txt`

The successful retrieval of the root flag confirmed full compromise of the target system.

9. Remediation Summary

The vulnerabilities found that are recommended to be fixed are:

1. Broken Access Control via Client-Side Cookie Manipulation

Authorization decisions must never rely on client-controlled values such as cookies.

User roles must be validated server-side using trusted session data, and authorization checks must be enforced for every protected resource.

2. Insecure Direct Object Reference (IDOR)

Implement server-side authorization checks for every request accessing user-controlled resources.

The application must verify that the authenticated user is authorized to access the requested object.

Where appropriate, use indirect or non-sequential object identifiers to reduce resource enumeration.

3. File upload

Restrict uploads to the file types strictly required by the application.

Validate both file extensions and file content, rename uploaded files, store them outside the web root where possible, and disable script execution in upload directories.

4. Remote Code Execution (RCE)

Prevent uploaded files from being interpreted as executable server-side code.

Run web applications using dedicated low-privileged accounts, maintain all software and dependencies with current security patches, and apply network-level controls where appropriate.

5. Exposed credentials

Do not store credentials in application files that can be accessed through the web application.

Use secure secret-management mechanisms, apply the principle of least privilege, and rotate any credentials that have been exposed.

6. SUID Misconfiguration

Remove the SUID bit from /usr/bin/bugtracker unless it is strictly required:

- `chmod u-s /usr/bin/bugtracker`

Regularly review SUID binaries and remove unnecessary privileged executables.

7. PATH Hijacking

Privileged programs should invoke external executables using absolute paths, such as:

- `/bin/cat`

They should not rely on user-controlled environment variables such as `PATH`.

10. Appendix A : Finding severities

Each finding has been assigned a severity rating of critical, high, medium, low. The rating is based off of an assessment of the priority with which each finding should be viewed and the potential each one has on the confidentiality, integrity and availability of Oopsie's data.

Severity Definitions	
Rating	Severity rating definition
Critical	Allows full compromise of the system or unauthorized access to highly sensitive information. May result in remote code execution, administrative access, or complete control of the target.
High	Allows significant access to sensitive data or functionality, privilege escalation, or lateral movement. May require an additional condition or vulnerability to achieve full compromise.
Medium	May facilitate subsequent attacks or expose information useful to an attacker. Typically requires chaining with other vulnerabilities to achieve full compromise.
Low	Has limited impact or is primarily informational. Does not typically provide direct system access but may assist reconnaissance or enumeration.

11. Appendix B: Tools

- Nmap
- Curl
- WhatWeb
- Nikto
- Gobuster
- Burp Suite

12. Appendix C : Technical report

Detailed technical evidence, commands, HTTP requests, exploitation steps, and supporting screenshots are available in the accompanying technical report:

See the attached file:

- Oopsie_Technical_Report.pdf